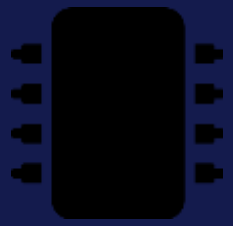


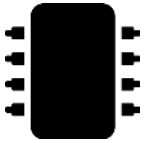
STORED PROGRAM CONCEPT

The Von Neumann Idea That Powers Every Modern Computer

UP LT Grade | Computer Science Mains | Descriptive Exam Prep

ABOUT ME

What Is It?



BEFORE (Hard-Wired Machines)

To run a new task, engineers had to physically rewire plugs and switches. Slow, error-prone, and inflexible.

AFTER (Von Neumann, 1945)

Program instructions and data are both stored in the same memory, in binary, so the computer's task can be changed just by loading a new program.

Exam-Ready Definition

"The program and its data are stored together in the same memory in binary form, and the CPU fetches, decodes, and executes instructions sequentially using a Program Counter."

Proposed by John von Neumann in 1945 • Basis of nearly all modern computers

Key Ideas to Write in Exam

1. **Shared Memory**
   Instructions and data live in the same memory — no separate storage for each.

2. **Binary Storage**
   Both are stored in binary; CPU tells them apart only by context (where PC points).

3. **Sequential Execution**
   Instructions run one after another in stored order, unless a jump/branch changes flow.

4. **Program Counter (PC)**
   The PC holds the address of the next instruction to fetch — drives the whole cycle.

5. **Self-Modifiable Code**
   Since code is just data in memory, a program can (in theory) modify itself.

Fetch – Decode – Execute Cycle

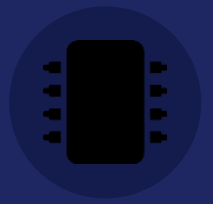
Why the Stored Program Concept makes this cycle possible

1. FETCH

CPU gets the instruction from memory at the address held by the Program Counter (PC).

2. DECODE

Control unit interprets the instruction — determines the operation and operands.



3. EXECUTE

ALU / control unit performs the operation; PC increments to the next address.

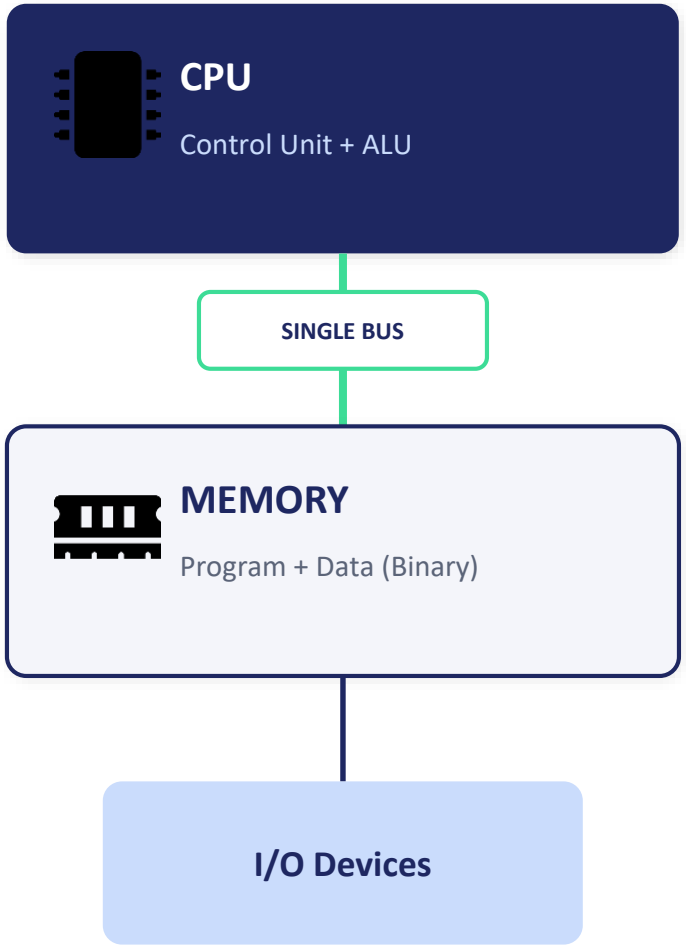
Cycle repeats continuously while the CPU is powered — this loop IS program execution.

Von Neumann Architecture

Single shared bus connects CPU and Memory

Remember

- CPU uses Program Counter (PC) to fetch next instruction
- Both instructions & data pass through the SAME bus
- This design underlies almost every modern computer



Von Neumann Bottleneck

Since one bus carries both instructions and data, the CPU cannot fetch both at the same time — this limits speed.

Solution: Harvard Architecture uses separate memories for instructions and data.

Summary: Advantages & Limitation

Advantages

- Simple, flexible design — reprogram by loading new instructions
- Easy to implement loops, branching, and subroutines
- No physical rewiring needed for new tasks
- Forms the basis of virtually all modern computers

Limitation

Single shared bus for instructions & data creates the Von Neumann Bottleneck — CPU can't fetch both simultaneously.

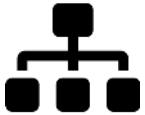
Leads to → Harvard Architecture

Separate memories for instructions and data allow simultaneous fetch — a strong comparison point for exams.

COMPONENTS OF A COMPUTER SYSTEM

The Building Blocks of Every Computer

What Makes Up a Computer System?



HARDWARE

Physical parts you can touch — CPU, memory, input/output devices, and storage.

SOFTWARE

Programs and instructions that tell the hardware what to do — system software & application software.

Exam-Ready Definition

"A computer system is an integrated set of hardware and software components — Input Unit, CPU, Memory Unit, and Output Unit — that work together to accept, process, store, and deliver data."

Together, hardware + software + data + users form the complete computer system.

Five Core Components

1

Input Unit

Accepts data & instructions from the outside world (keyboard, mouse, scanner) and converts to machine-readable form.

2

Central Processing Unit (CPU)

The “brain” — contains ALU, Control Unit, and Registers; fetches & executes instructions.

3

Memory Unit

Stores data and programs — divided into Primary (RAM/ROM) and Secondary (HDD/SSD) memory.

4

Output Unit

Converts processed results back into human-readable form (monitor, printer, speaker).

5

Bus / Interconnection System

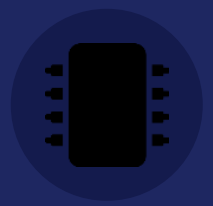
Address, Data, and Control buses that carry signals between all components.

Data Flow Through the System

How information moves from input to output

1. INPUT

Raw data entered via keyboard, mouse, or sensors.



2. PROCESS

CPU performs arithmetic/logic operations on the data.

3. STORE

Intermediate & final results held in memory.

4. OUTPUT

Processed result displayed or printed for the user.

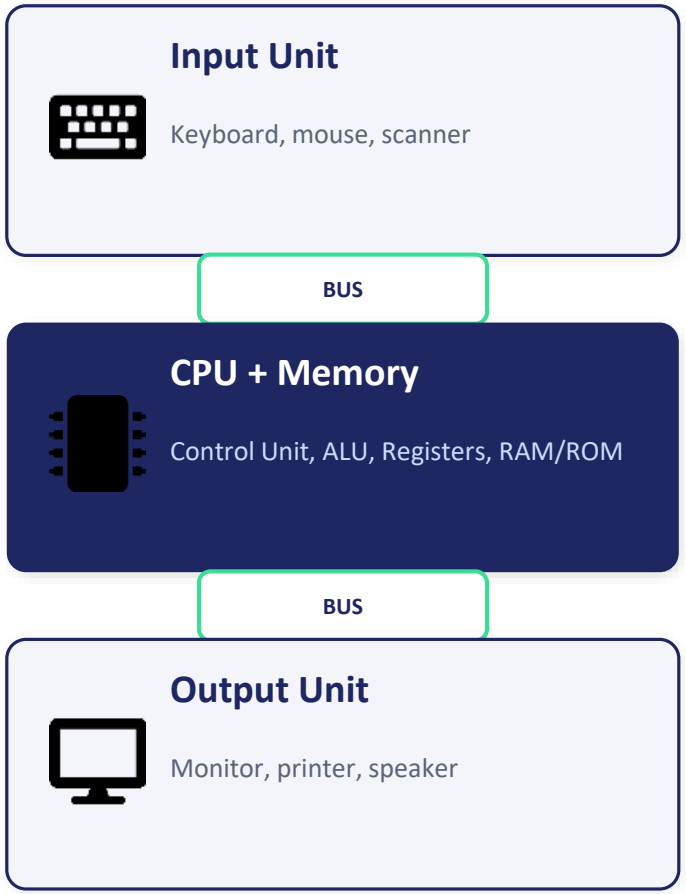
This is the classic IPOS cycle: Input → Process → Output → Storage.

System Architecture

How components connect via the system bus

Remember

- CPU is the central processing hub
- All units communicate via buses
- Memory holds both program & data (Stored Program Concept)



Exam Tip

Draw this block diagram first in any “Components of Computer System” question, then explain each block in 1-2 lines.

Hardware vs Software — Quick Recap

Hardware

- Physical, tangible components
- CPU, RAM, keyboard, monitor, hard disk
- Fails due to physical damage/wear
- Speed limited by physical design

Software

Set of instructions/programs that control hardware — System Software (OS) and Application Software (MS Word, browsers, etc.).

Key Point for Exam

Hardware executes; software instructs. Neither is useful without the other — this synergy defines a computer system.

MACHINE INSTRUCTION, OPCODES & OPERANDS

The Language the CPU Actually Understands

What Is a Machine Instruction?

OPCODE (Operation Code)

Specifies WHAT operation to perform — e.g. ADD, SUB, MOV, LOAD, STORE.

OPERAND

Specifies WHICH data to operate on — can be a register, memory address, or a constant value.

Exam-Ready Definition

"A machine instruction is a binary-encoded command, made of an opcode (the operation) and one or more operands (the data/addresses), that the CPU can directly fetch and execute."

Example: ADD R1, R2 → Opcode = ADD, Operands = R1 and R2

Key Points to Remember

1

Opcode Field

Tells the CPU which operation (add, subtract, jump, load) to perform; decoded by the Control Unit.

2

Operand Field(s)

Holds the data or address(es) the operation works on — can be 0, 1, 2, or 3 operands.

3

Addressing Mode

Part of the instruction that specifies HOW to interpret the operand (direct, indirect, immediate, etc.).

4

Register Operand

Fastest — operand value is already inside a CPU register.

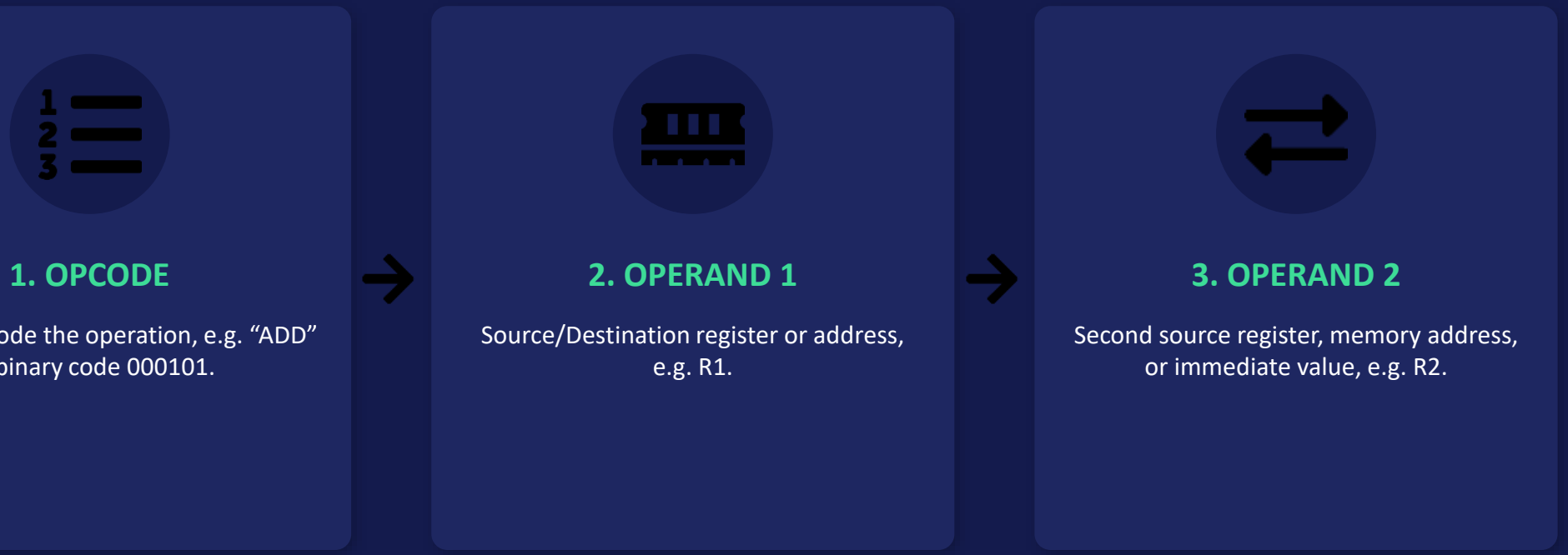
5

Memory / Immediate Operand

Memory operand needs a memory access; immediate operand is a constant embedded in the instruction itself.

Anatomy of a Machine Instruction

Example: ADD R1, R2 (Register-Register instruction)



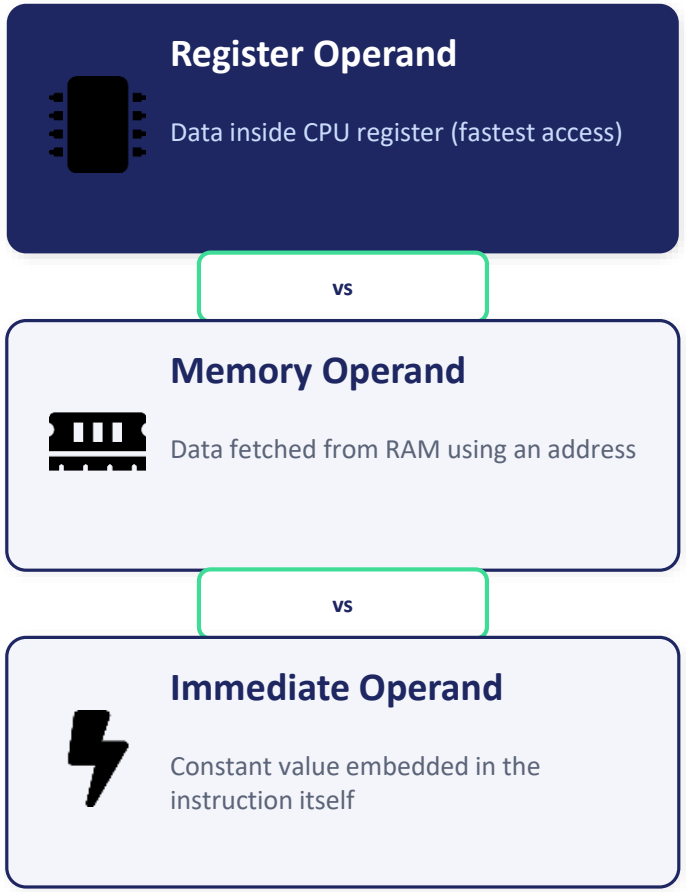
Total instruction length = Opcode bits + Operand bits (fixed in RISC, variable in CISC).

Operand Types

Where the actual data can live

Remember

- Register operands = fastest, no memory access
- Immediate operands need no fetch at all
- Memory operands are slowest due to bus access



Exam Tip

Always give one example instruction and label opcode vs operand explicitly — examiners award marks for correct labeling.

Instruction Length: Fixed vs Variable

Fixed-Length (RISC style)

- Every instruction is the same size (e.g. 32-bit)
- Simple, fast decoding by hardware
- Easier pipelining
- May waste bits for simple instructions

Variable-Length (CISC style)

Instruction size changes based on operands/addressing mode used (e.g. x86). More compact code but harder, slower decoding.

Exam Comparison Point

RISC trades code density for speed and simplicity; CISC trades decoding complexity for compact, powerful instructions.

INSTRUCTION CYCLE

The Complete Life of One Instruction Inside the CPU

What Is the Instruction Cycle?

MACHINE CYCLE

One complete pass of fetch, decode, and execute for a single instruction.

REPEATS CONTINUOUSLY

As long as the CPU has power and instructions to run, this cycle repeats — this repetition IS program execution.

Exam-Ready Definition

"The instruction cycle is the sequence of steps — Fetch, Decode, Execute, and (if needed) Interrupt Check — that the CPU repeats for every instruction it processes."

Also called the Fetch-Decode-Execute cycle, extended with an Interrupt-check phase in real CPUs.

Phases of the Instruction Cycle

1. **Fetch**
   CPU reads the instruction from memory at the address in the Program Counter (PC); PC then increments.

2. **Decode**
   Control Unit interprets the opcode to know which operation and operands are involved.

3. **Operand Fetch**
   If needed, the CPU fetches operand values from registers or memory.

4. **Execute**
   ALU or Control Unit performs the actual operation (add, move, compare, etc.).

5. **Interrupt Check**
   CPU checks for pending interrupts before moving to the next fetch cycle.

Fetch – Decode – Execute – Interrupt

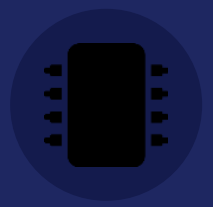
The four-phase view used in most textbooks

1. FETCH

Instruction copied from memory into the Instruction Register (IR).

2. DECODE

Control unit identifies opcode & operand locations.



3. EXECUTE

ALU performs the operation; result stored back.

4. INTERRUPT

CPU checks/services any pending interrupt signal.

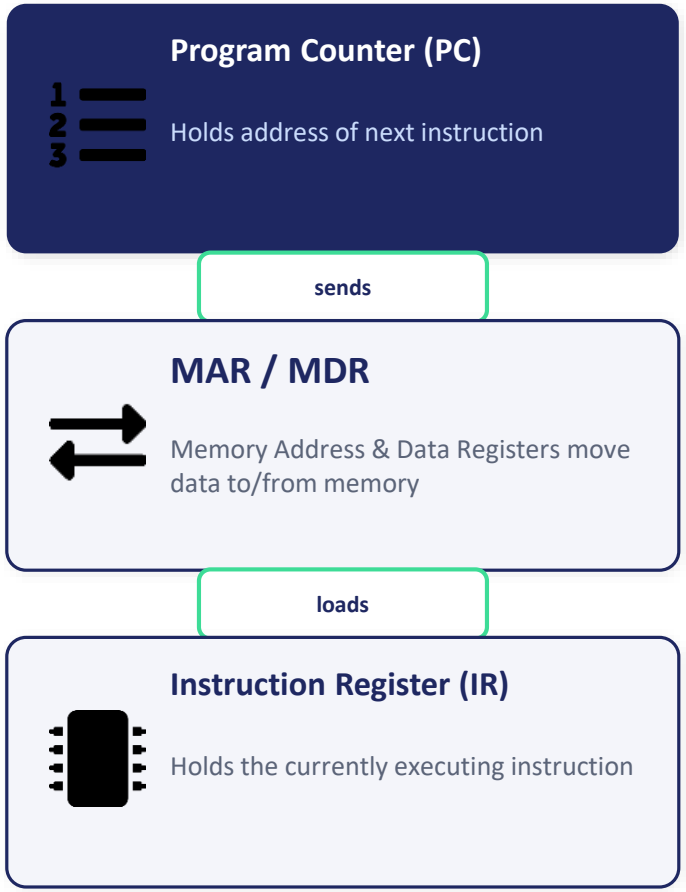
PC always points to the NEXT instruction — it's updated during the Fetch phase.

Registers Involved in the Cycle

How PC, IR, MAR, MDR interact with memory

Remember

- MAR = Memory Address Register
- MDR = Memory Data/Buffer Register
- IR holds instruction being decoded/executed



Exam Tip

Draw the register transfer notation, e.g. MAR ← PC; MDR ← M[MAR]; IR ← MDR — examiners love RTN steps.

Instruction Cycle vs Machine Cycle vs Clock Cycle

Clock Cycle

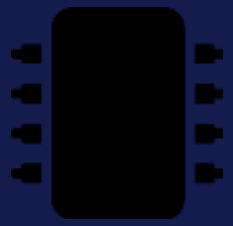
- Smallest unit of CPU time (one clock pulse)
- Basic timing unit for all operations
- Multiple clock cycles = 1 machine cycle

Machine & Instruction Cycle

A Machine Cycle is one memory access (fetch OR execute); an Instruction Cycle is the FULL fetch+decode+execute sequence, made of several machine cycles.

Exam Comparison Point

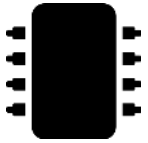
Hierarchy: Clock Cycle < Machine Cycle < Instruction Cycle. Always mention this nesting for full marks.



ORGANIZATION OF CPU & ALU

Inside the Brain of the Computer

What Is the CPU?



CPU (Central Processing Unit)

The main processing hardware — fetches, decodes, and executes instructions; made of ALU, CU, and Registers.

ALU (Arithmetic Logic Unit)

The calculating part of the CPU — performs all arithmetic (add, sub) and logical (AND, OR, NOT, compare) operations.

Exam-Ready Definition

"The CPU is organized into three main parts — the ALU for computation, the Control Unit for coordination, and Registers for fast temporary storage — all connected by internal buses."

The ALU never acts alone; the Control Unit tells it exactly which operation to perform and when.

Key Parts of the CPU

1

Arithmetic Logic Unit (ALU)

Executes arithmetic (+, −, ×) and logic (AND, OR, NOT, XOR, comparisons) operations.

2

Control Unit (CU)

Generates timing & control signals that direct all other components — decides WHEN and WHAT happens.

3

Registers

Small, ultra-fast storage locations inside the CPU (Accumulator, PC, IR, MAR, MDR, general-purpose registers).

4

Internal Bus

Connects ALU, CU, and registers so data can move between them within the CPU.

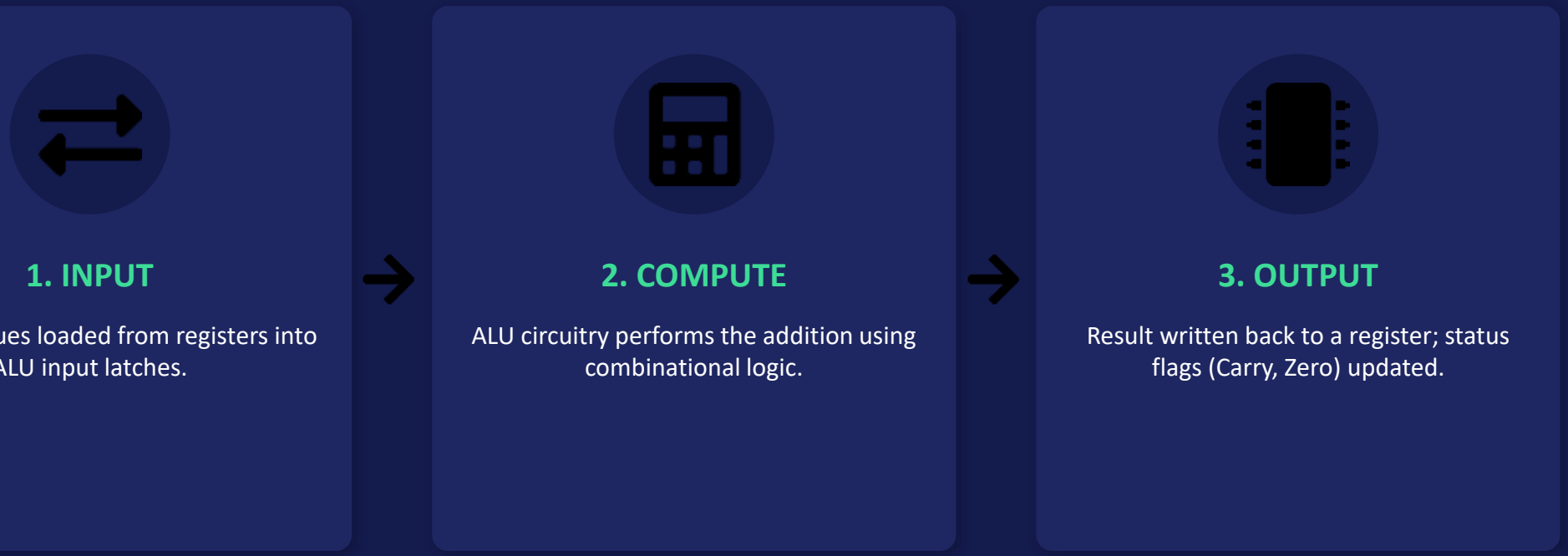
5

Flags / Status Register

Stores condition flags (Zero, Carry, Sign, Overflow) set after ALU operations.

How the ALU Performs an Operation

Example: executing ADD R1, R2



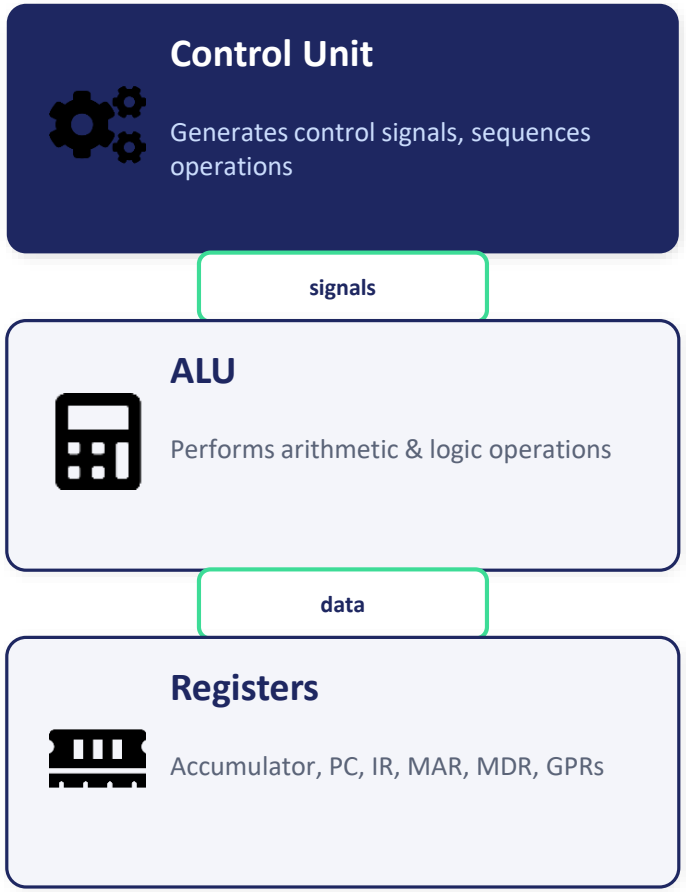
The Control Unit supplies the ALU's operation-select signal to choose ADD, SUB, AND, OR, etc.

CPU Internal Organization

How ALU, CU, and Registers connect

Remember

- CU = the “manager”, ALU = the “calculator”
- Registers act as the CPU's own scratchpad
- All three are linked by an internal data bus



Exam Tip

Mention that ALU operations set Flags (Zero/Carry/Sign/Overflow) — examiners frequently ask about status flags separately.

ALU Operations: Arithmetic vs Logical

Arithmetic Operations

- Addition, Subtraction
- Multiplication, Division (in advanced ALUs)
- Increment / Decrement
- Sets Carry & Overflow flags

Logical Operations

AND, OR, NOT, XOR, shift, and compare operations — used for bit manipulation, masking, and decision-making.

Exam Comparison Point

Arithmetic ops work on numeric values; logical ops work bit-by-bit — both go through the same ALU hardware.

HARDWIRED & MICROPROGRAMMED CONTROL UNIT

Two Ways to Generate CPU Control Signals

What Does a Control Unit Do?

HARDWIRED CONTROL UNIT

Control signals generated using fixed logic circuits (combinational + sequential logic gates).

MICROPROGRAMMED CONTROL UNIT

Control signals generated by executing “microinstructions” stored in a Control Memory (ROM).

Exam-Ready Definition

“The Control Unit generates the timing and control signals needed to execute instructions; it can be built as fixed hardware (hardwired) or as a small stored program (microprogrammed).”

Choice affects speed, flexibility, and cost of the CPU design.

Key Points to Remember

1. **Hardwired CU**
   Uses logic gates, flip-flops, and decoders — very fast but difficult to modify once built.

2. **Microprogrammed CU**
   Uses a Control Memory storing microinstructions (microcode) — slower but easily updated/redesigned.

3. **Control Word**
   Each microinstruction is a “control word” specifying which control signals to activate.

4. **Sequencer**
   In microprogrammed CU, decides the address of the next microinstruction to execute.

5. **Used In**
   Hardwired → RISC processors (speed-critical); Microprogrammed → CISC processors (complex instruction sets).

How Microprogrammed Control Works

Steps from opcode to control signals

1. OPCODE IN

Instruction opcode maps to a starting address in Control Memory.

2. FETCH MICROINSTR.

Microinstruction is read from Control Memory (a small, fast ROM).

3. GENERATE SIGNALS

Control word bits directly activate the needed control lines.

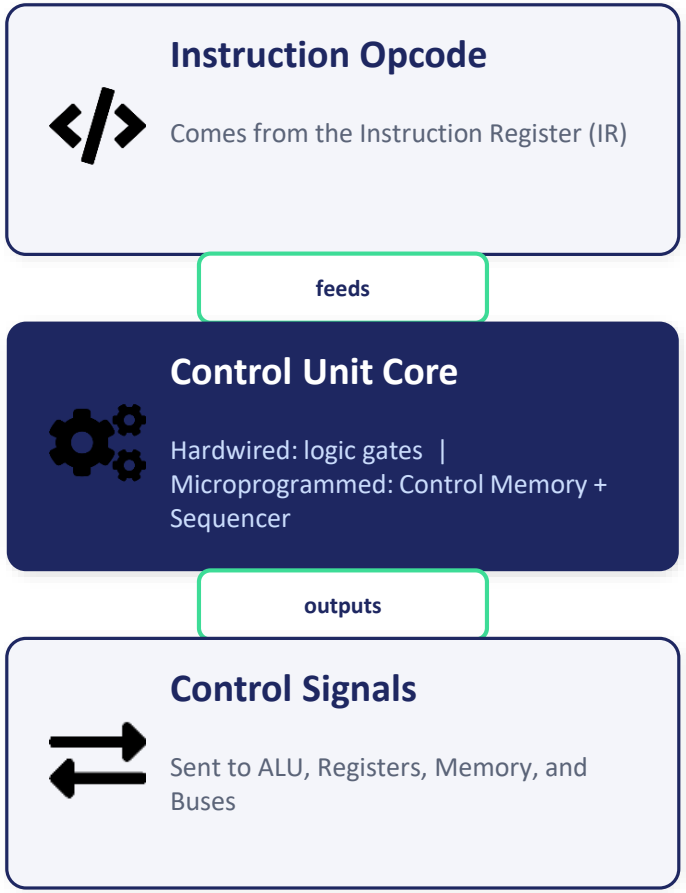
This repeats until the full sequence of micro-operations for that instruction is complete.

Hardwired vs Microprogrammed — Structure

Where control signals come from

Remember

- Hardwired = speed, low flexibility
- Microprogrammed = flexibility, easier debugging
- Both ultimately generate the same type of control signals



Exam Tip

Draw a simple two-box diagram (Opcode → CU → Signals) and label which type it is — this alone earns diagram marks.

Hardwired vs Microprogrammed — Full Comparison

Hardwired CU

- Faster (pure hardware logic)
- Difficult to modify/redesign
- Cheaper for simple instruction sets
- Best for RISC processors

Microprogrammed CU

Slower than hardwired (extra memory access per micro-op) but far easier to modify — just change the microcode in Control Memory.

Exam Comparison Point

Best for CISC processors with large, complex instruction sets where flexibility matters more than raw speed.

GENERAL & SPECIAL PURPOSE REGISTERS

The CPU's Fastest Storage Locations

What Is a Register?

GENERAL PURPOSE REGISTERS (GPR)

Multi-purpose registers (R0, R1, R2…) usable by the programmer/compiler to hold any data or address.

SPECIAL PURPOSE REGISTERS (SPR)

Reserved for specific CPU functions — PC, IR, MAR, MDR, Accumulator, Status/Flag register.

Exam-Ready Definition

"Registers are small, extremely fast storage units inside the CPU — General Purpose Registers hold user data/addresses flexibly, while Special Purpose Registers are dedicated to specific control functions."

Registers are the fastest memory in the entire computer — faster than cache or RAM.

Important Special Purpose Registers

1

Program Counter (PC)

Holds the address of the NEXT instruction to be fetched.

2

Instruction Register (IR)

Holds the instruction currently being decoded/executed.

3

Memory Address Register (MAR)

Holds the address of the memory location to be accessed.

4

Memory Data Register (MDR)

Holds the data being transferred to/from memory.

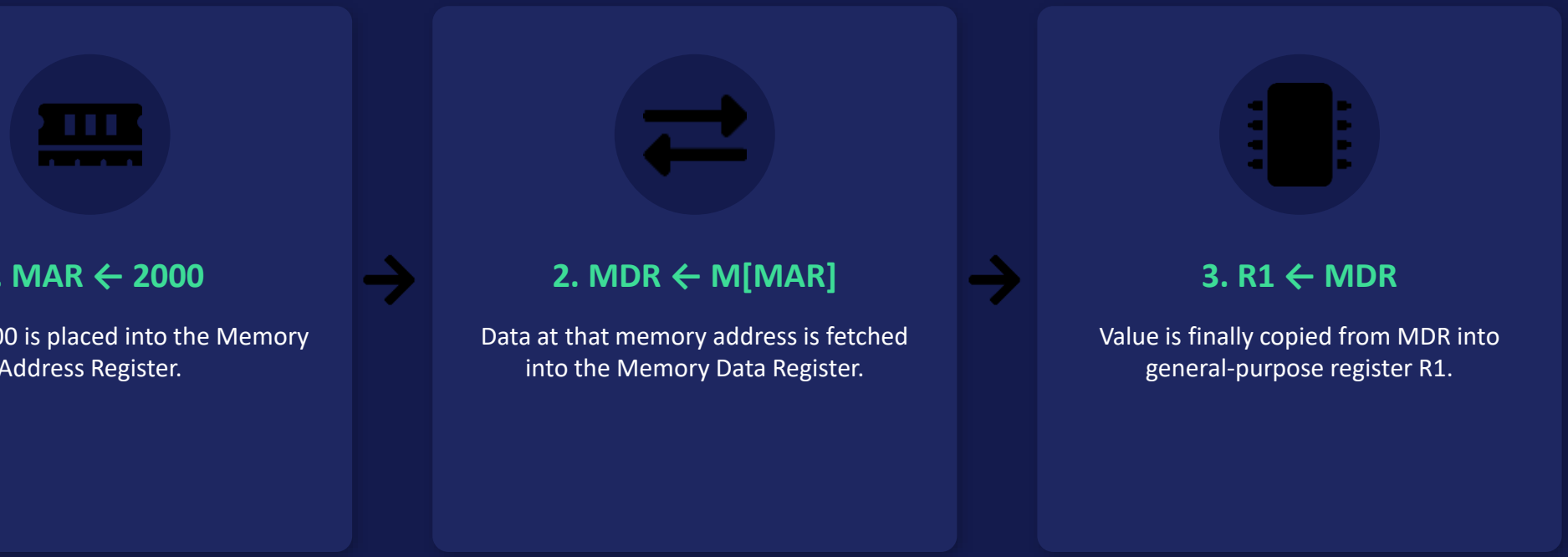
5

Accumulator (ACC) & Status Register

Accumulator stores intermediate ALU results; Status register holds condition flags (Zero, Carry, Sign, Overflow).

Register Involvement During Execution

Example: LOAD R1, [2000]



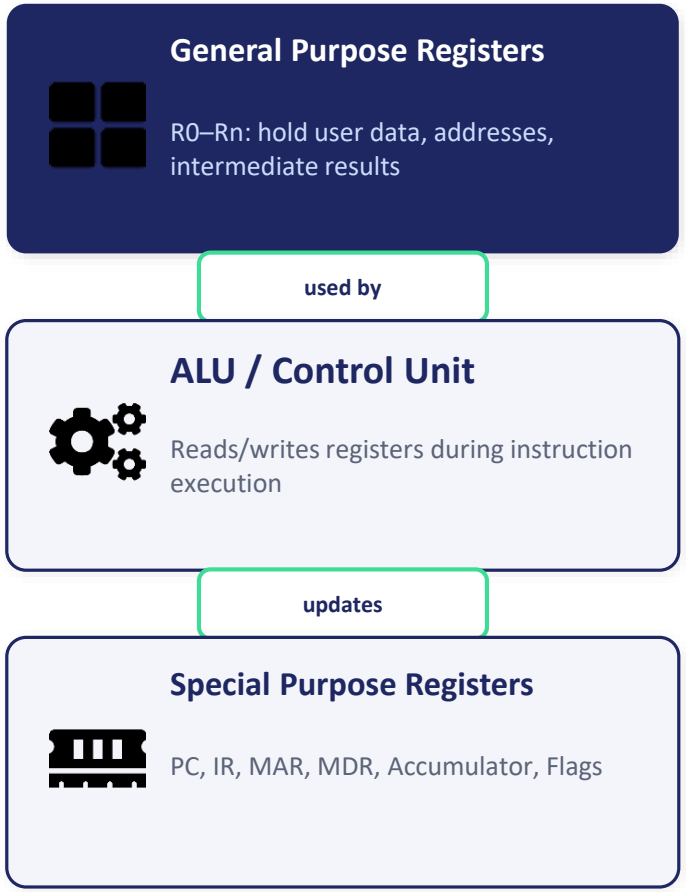
This register-transfer notation (RTN) is a favorite descriptive-exam question.

Register File Organization

How GPRs and SPRs sit inside the CPU

Remember

- GPRs are named/numbered (R0, R1…)
- SPRs each have ONE fixed job
- Number of GPRs varies by architecture (8, 16, 32…)



Exam Tip

List at least 4 SPRs with their exact function — this is one of the most repeated descriptive questions in COA papers.

GPR vs SPR — Quick Comparison

General Purpose Registers

- Usable for any data/address by programmer
- Named R0, R1, R2… Rn
- Compiler decides what to store where
- Examples: R0–R15 in ARM

Special Purpose Registers

Each has ONE dedicated role fixed by CPU design — programmer cannot repurpose them (e.g., PC always holds next instruction address).

Exam Comparison Point

GPRs = flexibility for the programmer; SPRs = essential plumbing for the fetch-decode-execute cycle itself.

MEMORY ORGANIZATION

The Memory Hierarchy That Balances Speed, Size & Cost

Why Organize Memory in Layers?

THE PROBLEM

Fast memory (registers, cache) is expensive & small; large memory (disk) is cheap but slow. No single memory type is perfect.

THE SOLUTION

A Memory Hierarchy — layering multiple memory types from fastest/smallest to slowest/largest.

Exam-Ready Definition

"Memory organization arranges storage into a hierarchy — Registers, Cache, Primary (RAM), and Secondary Memory — balancing speed, capacity, and cost so the CPU gets data as fast as possible."

General rule: speed decreases and capacity increases as you go down the hierarchy.

Levels of the Memory Hierarchy

1. **Registers**
   Inside the CPU; fastest, smallest (bytes); zero access delay.

2. **Cache Memory (L1/L2/L3)**
   Very fast SRAM between CPU and RAM; stores frequently used data to reduce access time.

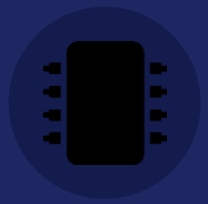
3. **Primary Memory (RAM/ROM)**
   Main working memory; volatile RAM holds running programs, non-volatile ROM holds firmware.

4. **Secondary Memory**
   Hard disks, SSDs — large capacity, non-volatile, but much slower than RAM.

5. **Tertiary/Backup Memory**
   Magnetic tapes, cloud backups — used for archival, rarely accessed directly by CPU.

How Data Moves Down the Hierarchy

On a cache miss, data is pulled up from lower levels



1. CPU REQUESTS

CPU asks for a data value needed by the current instruction.

2. CHECK CACHE

If found (cache hit) — instantly returned.
If not (miss) — go to RAM.

3. FETCH FROM RAM/DISK

Data block is loaded from RAM (or disk) into cache for future fast access.

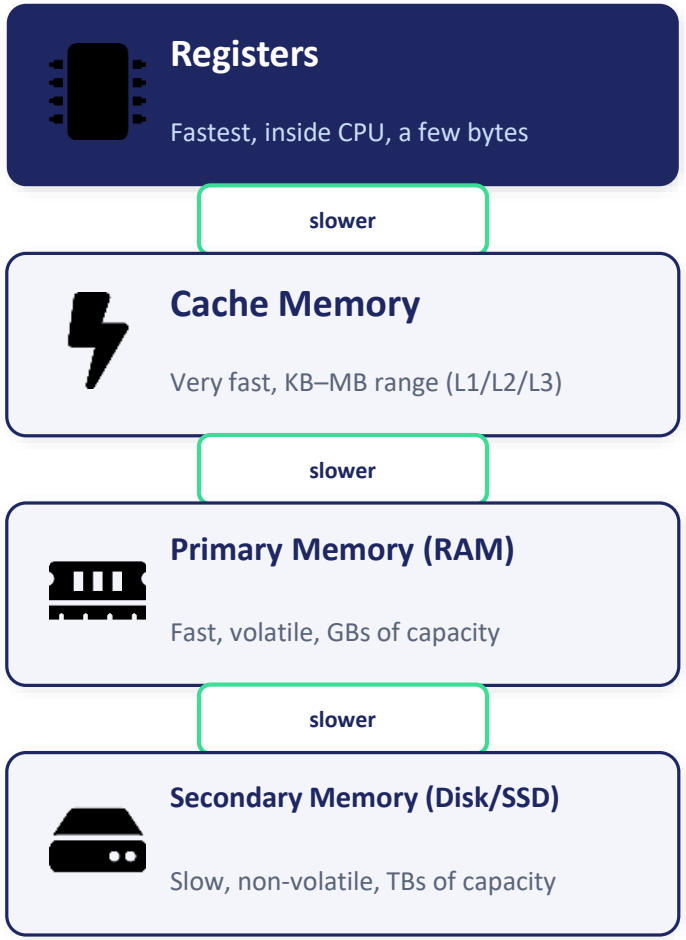
Principle of Locality (temporal & spatial) is why caching works so well in practice.

The Memory Hierarchy Pyramid

Fastest & smallest at top, slowest & largest at bottom

Remember

- Speed ↓ as you go down
- Capacity ↑ as you go down
- Cost per byte ↓ as you go down



Exam Tip

Draw this pyramid and label speed/cost/capacity trends on the side — a guaranteed scoring diagram.

RAM vs ROM

RAM (Random Access Memory)

- Volatile — loses data on power-off
- Read AND write access
- Holds running programs & OS
- Types: SRAM (cache), DRAM (main memory)

ROM (Read-Only Memory)

Non-volatile — retains data without power; mainly read-only, stores firmware/BIOS/bootstrap loader.

Exam Comparison Point

Variants of ROM: PROM, EPROM, EEPROM — each differs in how/whether it can be reprogrammed.

I/O ORGANIZATION

How the CPU Talks to the Outside World

What Is I/O Organization?

THE CHALLENGE

I/O devices (keyboard, disk, printer) are far slower and use different signal formats than the CPU.

THE SOLUTION

I/O interfaces & controllers bridge the speed and format gap, using defined data-transfer techniques.

Exam-Ready Definition

"I/O Organization deals with how the CPU communicates with input/output devices — using Programmed I/O, Interrupt-driven I/O, or DMA — through I/O interfaces and controllers."

Choice of technique trades off CPU involvement against transfer speed.

Data Transfer Techniques

1

Programmed I/O

CPU continuously polls the device status and transfers data itself — simple but wastes CPU time.

2

Interrupt-Driven I/O

Device sends an interrupt signal when ready; CPU handles other tasks meanwhile — more efficient.

3

Direct Memory Access (DMA)

A DMA controller transfers data directly between I/O device and memory, bypassing the CPU.

4

I/O Interface / Controller

Hardware that matches device speed & signal format to the system bus.

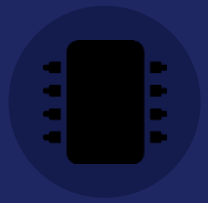
5

I/O Addressing

Memory-Mapped I/O (devices share address space with memory) vs Isolated/Port-Mapped I/O (separate address space).

DMA Transfer — Step by Step

How data moves without CPU involvement



1. CPU INITIATES

CPU sets up DMA controller with source, destination, and count, then continues other work.

2. DMA TRANSFERS

DMA controller directly moves data between I/O device and memory over the bus.

3. INTERRUPT ON DONE

DMA controller signals CPU via interrupt once the transfer is complete.

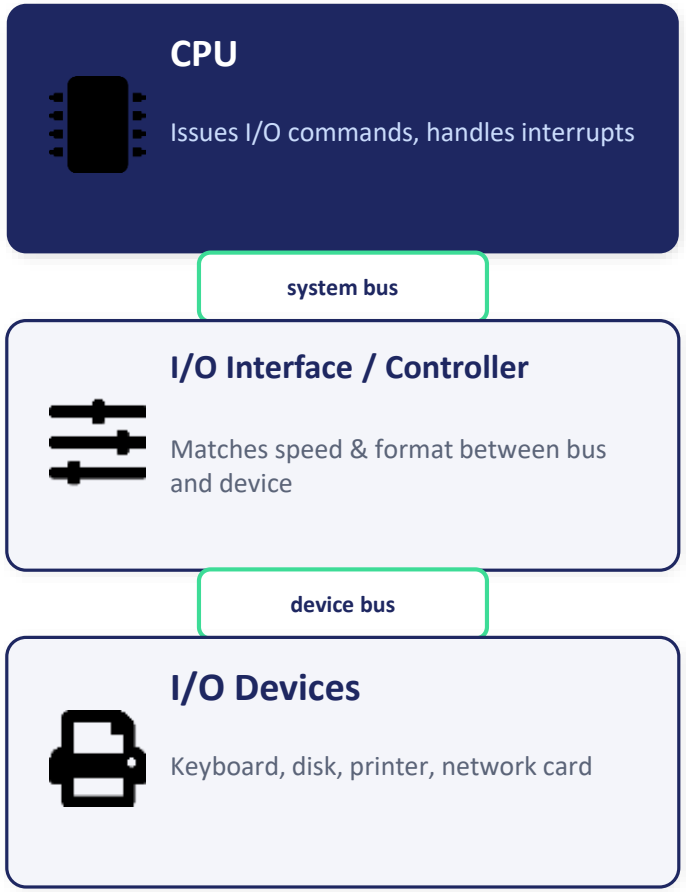
DMA drastically reduces CPU overhead for large, fast data transfers (e.g. disk-to-memory).

I/O System Architecture

How devices connect to the CPU and memory

Remember

- Programmed I/O wastes CPU cycles
- Interrupt-driven frees CPU while waiting
- DMA is fastest for bulk transfers



Exam Tip

Always compare all three transfer methods (Programmed/Interrupt/DMA) together — a very common 5-mark question.

Interrupt-Driven I/O vs DMA

Interrupt-Driven I/O

- CPU issues request, does other work
- Device interrupts CPU when ready
- CPU itself transfers each data word
- Good for slow, small transfers (keyboard)

DMA (Direct Memory Access)

A dedicated DMA controller moves entire data blocks directly between device and memory — CPU is interrupted only once, at completion.

Exam Comparison Point

DMA is far more efficient for large transfers (disk, video) since CPU isn't tied up moving each word.

FUNCTIONING OF CPU

How the Processor Actually Runs a Program

How Does the CPU “Run” a Program?

THE BIG IDEA

A program is just a sequence of instructions in memory; the CPU repeatedly fetches, decodes, and executes them one by one.

COORDINATION

The Control Unit synchronizes ALU, Registers, Memory, and I/O using a system clock, so every step happens at the right time.

Exam-Ready Definition

“The CPU functions by continuously executing the Fetch-Decode-Execute cycle, using its Control Unit to coordinate the ALU, registers, memory, and I/O — driven by the system clock and Program Counter.”

Everything a computer “does” ultimately reduces to this loop running extremely fast (billions of times per second).

How the Pieces Work Together

1. **System Clock**
   Generates timing pulses that synchronize every operation inside the CPU.

2. **Program Counter (PC)**
   Always points to the address of the next instruction to fetch.

3. **Control Unit**
   Interprets the opcode and issues control signals to the right components at the right time.

4. **ALU + Registers**
   Perform the actual computation and hold data during processing.

5. **Buses**
   Address, Data, and Control buses carry information between CPU, memory, and I/O throughout execution.

One Full Pass Through the CPU

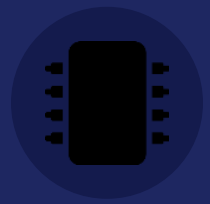
A simplified functional view

1. FETCH

Instruction fetched from memory using PC; PC incremented.

2. DECODE & CONTROL

Control Unit decodes opcode and generates control signals.



3. EXECUTE & STORE

ALU executes; result written to register/memory; flags updated.

This is functionally identical to the Instruction Cycle — viewed from the "whole CPU" perspective.

Functional Block View of the CPU

All parts working under one synchronized clock

Remember

- Clock speed (GHz) = how fast this loop repeats
- CU is the “conductor” of the whole orchestra
- Buses are the communication highways

Exam Tip

Link this topic to Instruction Cycle & CPU Organization in your answer — examiners like seeing connections across topics.

Single-Cycle vs Multi-Cycle CPU Functioning

Single-Cycle Design

- Every instruction takes exactly 1 clock cycle
- Simple control logic
- Clock must be slow (limited by slowest instruction)
- Wastes time for simple instructions

Multi-Cycle Design

Instructions take a variable number of clock cycles depending on complexity, allowing a faster overall clock rate.

Exam Comparison Point

Modern CPUs use pipelining (multi-cycle, overlapped) for much higher throughput than either simple design.

INSTRUCTION FORMATS

How Bits Inside an Instruction Are Laid Out

What Is an Instruction Format?

DEFINITION

The layout/arrangement of bits within a machine instruction — which bits are opcode, which are operand(s).

WHY IT MATTERS

Determines instruction length, number of addressable operands, and how much memory/registers can be addressed.

Exam-Ready Definition

"An instruction format specifies the number of bits allocated to the opcode and to each operand field, defining how many addresses (0, 1, 2, or 3) an instruction can directly reference."

Format design is a trade-off between instruction length, addressing range, and hardware simplicity.

Common Instruction Format Types

1

Zero-Address Format

No explicit operand fields; uses a stack (push/pop) — used in stack-based machines.

2

One-Address Format

Uses an implied Accumulator as the second operand, e.g. ADD X means ACC = ACC + X.

3

Two-Address Format

One operand doubles as source and destination, e.g. ADD R1, R2 means R1 = R1 + R2.

4

Three-Address Format

Separate fields for two sources and one destination, e.g. ADD R1, R2, R3 means R1 = R2 + R3.

5

Opcode Field Size

Determines max distinct instructions possible — n bits give up to 2^n operations.

Bit Layout Comparison

Same ADD operation in different formats

1. 0-ADDRESS

[OPCODE] — operands implied on a stack (e.g. PUSH/ADD/POP).

2. 1-ADDRESS

[OPCODE | ADDRESS] — second operand is always the Accumulator.

3. 2 / 3-ADDRESS

[OPCODE | ADDR1 | ADDR2 | (ADDR3)] — explicit operand fields.

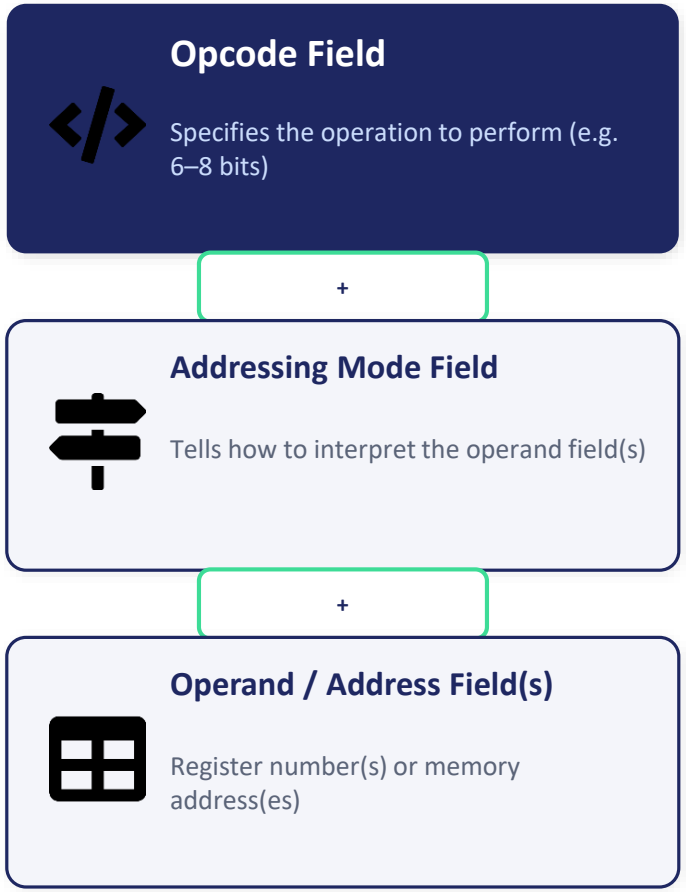
More address fields = more flexibility per instruction, but longer instruction length.

General Instruction Format Layout

How a single instruction word is divided

Remember

- Total instruction length = sum of all fields
- More operand fields → fewer instructions needed
- Fewer fields → shorter, denser instructions



Exam Tip

Draw the bit-field boxes with sample bit-widths (e.g. 8|4|4|4 bits) — numeric examples earn extra marks.

Two-Address vs Three-Address Format

Two-Address Format

- Shorter instructions (fewer bits)
- One operand is overwritten by result
- Needs extra MOV instructions to preserve data
- Common in x86 (CISC)

Three-Address Format

Separate destination and two source operands mean original values are preserved — fewer total instructions needed for complex expressions.

Exam Comparison Point

Three-address formats are common in RISC machines (e.g. ARM, MIPS) and simplify compiler code generation.

INSTRUCTION TYPES

Categorizing What Instructions Can Do

How Are Instructions Classified?

BY FUNCTION

Instructions are grouped by the KIND of task they perform — data movement, arithmetic, logic, control, or I/O.

WHY CLASSIFY

Helps in designing instruction sets, writing compilers, and understanding CPU capability at a glance.

Exam-Ready Definition

"Instruction types are broad categories — Data Transfer, Arithmetic, Logical, Control Transfer, and Input/Output — that together define everything a CPU's instruction set can do."

Every machine-level program is just a sequence of instructions drawn from these categories.

The Five Major Instruction Types

1. **Data Transfer Instructions**
   Move data between registers, memory, and I/O — e.g. MOV, LOAD, STORE, PUSH, POP.

2. **Arithmetic Instructions**
   Perform math operations — e.g. ADD, SUB, MUL, DIV, INC, DEC.

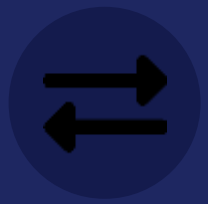
3. **Logical Instructions**
   Perform bit-level operations — e.g. AND, OR, NOT, XOR, shift, rotate.

4. **Control Transfer Instructions**
   Change the normal execution sequence — e.g. JMP, CALL, RET, conditional branches (JZ, JNZ).

5. **Input/Output Instructions**
   Transfer data between CPU and external devices — e.g. IN, OUT.

Example: A Small Program's Instruction Mix

MOV, ADD, CMP, JZ used together



1. DATA TRANSFER

MOV R1, 5 — loads value 5 into register R1.

2. ARITHMETIC

ADD R1, R2 — adds R2 into R1.

3. CONTROL TRANSFER

JZ LABEL — jumps to LABEL if the result is zero.

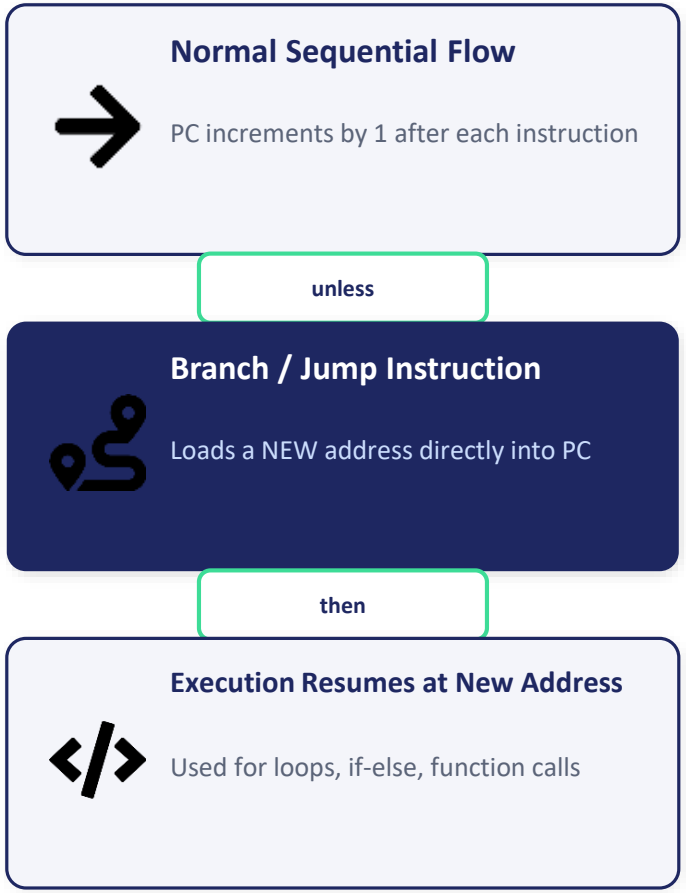
Real programs mix instruction types constantly — rarely just one type in isolation.

Control Transfer — Special Focus

How branching changes the normal PC sequence

Remember

- Unconditional jump: always taken (JMP)
- Conditional jump: taken only if flag matches (JZ, JC)
- CALL/RET manage subroutine returns via a stack



Exam Tip

Give at least one real instruction example per type — examiners specifically reward concrete mnemonics like MOV, ADD, JZ.

Conditional vs Unconditional Branching

Unconditional Branch

- Always transfers control (e.g. JMP)
- Used for loops, function calls (CALL)
- No flag/condition checked
- Simple, predictable

Conditional Branch

Transfers control only if a specific status flag matches (e.g. JZ jumps if Zero flag = 1) — basis of if-else and loop logic.

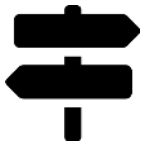
Exam Comparison Point

Conditional branches are what make programs "decide" — without them, every program would run the same way every time.

ADDRESSING MODES

How the CPU Locates an Instruction's Operand

What Is an Addressing Mode?



DEFINITION

The method by which the effective address of an operand is determined from the instruction.

WHY IT MATTERS

Gives flexibility to access constants, registers, arrays, and pointers efficiently without needing separate instructions for each case.

Exam-Ready Definition

"An addressing mode defines the rule used to calculate the effective address of the operand referenced in an instruction — e.g. immediate, direct, indirect, or indexed."

The addressing mode is usually encoded as part of the instruction's opcode/mode field.

Common Addressing Modes

1

Immediate Addressing

Operand VALUE is given directly in the instruction — e.g. MOV R1, #5 (fastest, no memory access).

2

Direct Addressing

Instruction contains the actual memory ADDRESS of the operand — e.g. MOV R1, [2000].

3

Indirect Addressing

Instruction gives the address of a location that CONTAINS the operand's address (pointer).

4

Register Addressing

Operand is inside a CPU register — e.g. MOV R1, R2 (very fast, no memory access).

5

Indexed / Base Addressing

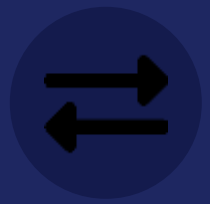
Effective address = Base register + Index/Offset — ideal for accessing arrays.

Finding the Effective Address

Direct vs Indirect vs Indexed — step by step

1. DIRECT

Address field IS the effective address — go straight to memory.



2. INDIRECT

Address field points to a memory cell that HOLDS the real address.

3. INDEXED

Effective Address = Base Register value + Offset in instruction.

Indirect addressing needs one EXTRA memory access compared to direct addressing.

Indirect Addressing — Visualized

One extra hop through memory to reach the data

Remember

- Immediate/Register modes = fastest (no extra memory access)
- Indirect mode = flexible but slower (extra fetch)
- Indexed mode is ideal for looping over arrays

Exam Tip

Draw the pointer-chasing diagram for Indirect addressing — it's the mode students most often explain incorrectly.

Direct vs Indirect Addressing

Direct Addressing

- Address field = effective address directly
- Only 1 memory access needed
- Limited address range (bits available)
- Simple and fast

Indirect Addressing

Address field points to a memory location holding the real address — 2 memory accesses needed, but supports a much larger effective address range.

Exam Comparison Point

Indirect addressing is essential for implementing pointers, linked lists, and dynamic data structures.

COMMON MICROPROCESSOR INSTRUCTIONS

The Everyday Toolkit of a Processor

What Counts as a “Common” Instruction?

DEFINITION

The small, frequently-used set of instructions found in nearly every microprocessor's instruction set (8085, 8086, ARM, etc.).

WHY LEARN THESE

These mnemonics appear directly in exam questions asking to “write an assembly program” or “identify the instruction.”

Exam-Ready Definition

“Common microprocessor instructions are the basic mnemonics — data movement, arithmetic, logic, and branching — that form the core vocabulary every assembly-level program is built from.”

Exact mnemonics vary slightly by processor family (Intel 8085/8086 vs ARM vs MIPS) but the categories stay the same.

Frequently Used Instruction Mnemonics

1. **MOV / LOAD / STORE**
   Move data between registers and memory — e.g. MOV A, B; LOAD R1, [2000]; STORE [3000], R1.

2. **ADD / SUB / INR / DCR**
   Arithmetic — addition, subtraction, increment, decrement, e.g. ADD B; INR A.

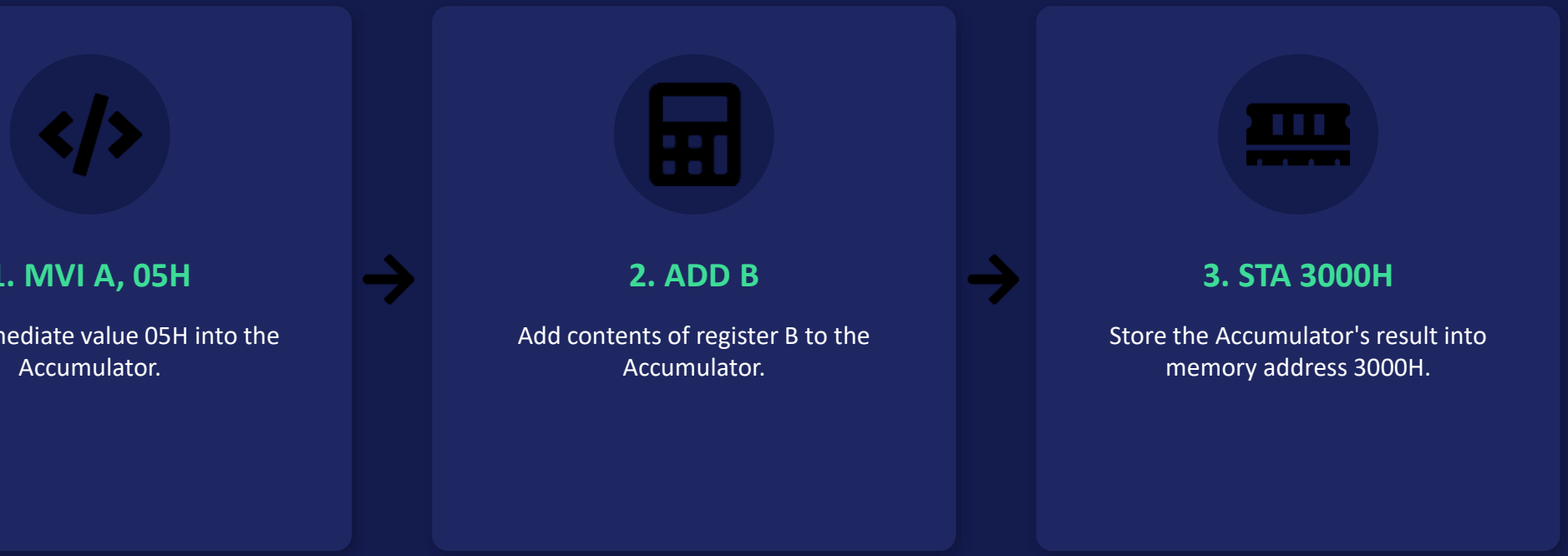
3. **AND / OR / XOR / CMP**
   Logical & comparison operations — e.g. CMP B compares Accumulator with B, sets flags.

4. **JMP / JZ / JNZ / CALL / RET**
   Control transfer — unconditional/conditional jumps, subroutine call & return.

5. **IN / OUT / HLT**
   I/O transfer and halting the processor — e.g. IN PORT1; HLT stops execution.

Tiny Program Example

Add two numbers and store the result (8085-style)



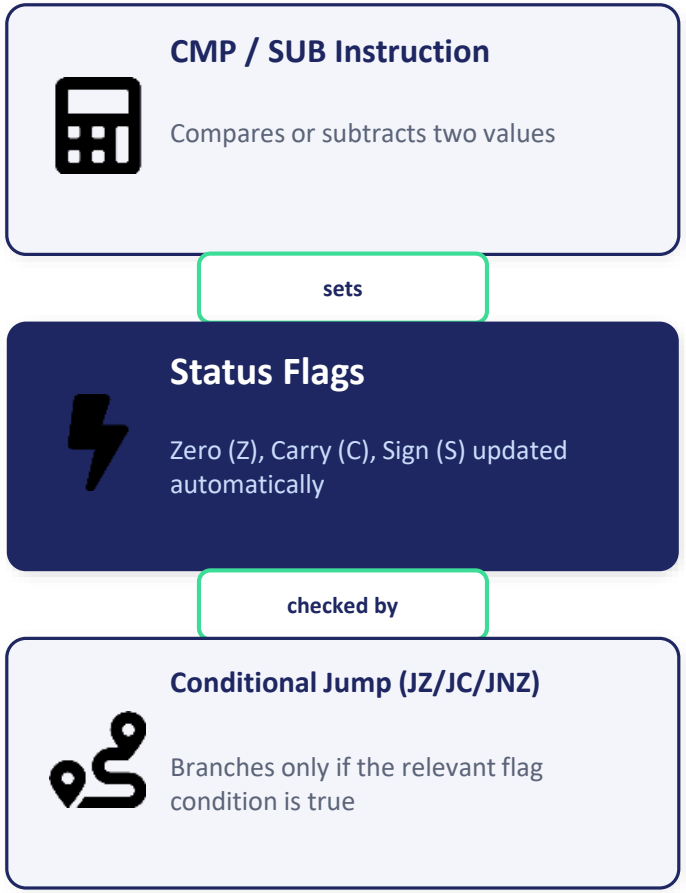
Tracing such short programs line-by-line is a very common descriptive-exam question.

Instruction → Flags → Branch Connection

How CMP/SUB feed into conditional jumps

Remember

- HLT stops the processor completely
- CALL pushes return address to stack
- RET pops it back to resume the caller



Exam Tip

Practice tracing 4–6 line assembly snippets and predicting register/flag values — this is a guaranteed question type.

CALL/RET vs JMP — Don't Confuse Them

JMP (Jump)

- Simply transfers control to a new address
- Does NOT save the return address
- Cannot easily “come back”
- Used for loops, simple branching

CALL / RET (Subroutine)

CALL saves the current PC (return address) on the stack before jumping; RET pops that address back into PC to resume exactly where it left off.

Exam Comparison Point

CALL/RET is how functions/subroutines work at the hardware level — JMP alone cannot support this reusability.

MULTI-CORE ARCHITECTURE

Many Processing Cores, One Chip

What Is a Multi-Core Processor?

SINGLE-CORE LIMIT

A single core can only execute one instruction stream at a time; increasing clock speed alone causes excess heat & power use.

MULTI-CORE SOLUTION

Multiple independent CPU cores are placed on a single chip, each capable of executing instructions in parallel.

Exam-Ready Definition

"A multi-core architecture integrates two or more independent processing cores onto a single chip, each with its own execution units and often its own cache, sharing memory and other resources to run tasks in parallel."

Popularized because raw clock-speed scaling ("more GHz") hit power & heat limits in the mid-2000s.

Key Features of Multi-Core Systems

1

Independent Cores

Each core has its own ALU, registers, and control unit — can execute a separate instruction stream.

2

Shared / Private Cache

Each core often has private L1 cache, while L2/L3 cache is typically shared across cores.

3

Parallelism

Multiple threads/processes run truly simultaneously, unlike time-slicing on a single core.

4

Inter-core Communication

Cores communicate via a shared bus or on-chip interconnect network.

5

Software Dependency

Real speedup requires multi-threaded software — single-threaded programs don't automatically benefit.

How a Task Gets Parallelized

Splitting one big job across cores

1. SPLIT TASK

OS/software divides a job into independent sub-tasks or threads.

2. ASSIGN TO CORES

Scheduler assigns each thread to a separate core for simultaneous execution.

3. MERGE RESULTS

Results from all cores are combined/synchronized into the final output.

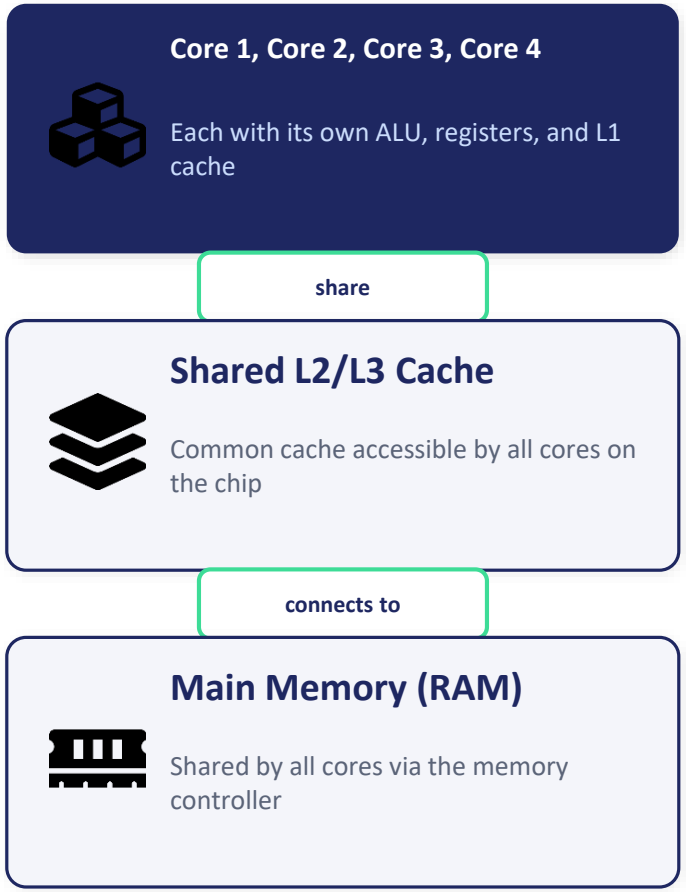
This is why video encoding, rendering, and compiling get dramatically faster on multi-core CPUs.

Multi-Core Chip Layout

Cores, private cache, shared cache, and memory

Remember

- Cores are on the SAME chip/die
- Contrast with multi-processor (separate chips)
- Examples: Dual-core, Quad-core, Octa-core CPUs



Exam Tip

Clearly distinguish Multi-core (one chip) from Multiprocessor (multiple chips) — examiners specifically test this confusion.

Multi-Core vs Increasing Clock Speed

Higher Clock Speed

- Single core made faster and faster
- Diminishing returns after ~3–4 GHz
- Excess heat & power consumption
- Hit a practical physical limit

Multi-Core (Modern Approach)

Instead of one super-fast core, place several moderate-speed cores on one chip — better performance-per-watt for parallel workloads.

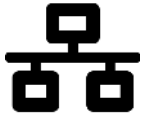
Exam Comparison Point

Multi-core trades single-thread speed for overall throughput — which is why multi-threaded software design matters today.

MULTIPROCESSOR & MULTICOMPUTER SYSTEMS

Scaling Computing Power Beyond One Chip

Two Ways to Combine Multiple Processors



MULTIPROCESSOR SYSTEM

Two or more CPUs share a COMMON memory and are controlled by a single operating system (tightly coupled).

MULTICOMPUTER SYSTEM

Multiple independent computers, each with its OWN memory, connected via a network (loosely coupled) — communicate by message passing.

Exam-Ready Definition

"A multiprocessor system tightly couples multiple CPUs around shared memory under one OS, while a multicomputer system loosely couples independent computers, each with private memory, communicating over a network."

Both aim to increase total computing power, but differ fundamentally in memory sharing and coupling.

Key Concepts to Remember

1

Shared Memory (Multiprocessor)

All processors can directly read/write the same physical memory — fast communication, but needs synchronization.

2

Message Passing (Multicomputer)

Each node has private memory; nodes exchange data only via explicit messages over a network.

3

Tightly vs Loosely Coupled

Multiprocessor = tightly coupled (shared memory, low latency); Multicomputer = loosely coupled (independent nodes, higher latency).

4

Symmetric vs Asymmetric Multiprocessing

SMP: all CPUs are equal & run OS copies; AMP: one master CPU controls, others are subordinate.

5

Examples

Multiprocessor: multi-socket servers; Multicomputer: computer clusters, distributed systems, cloud data centers.

How a Multicomputer Handles a Task

Distributed processing via message passing

1. TASK DIVIDED

A large job is split into independent parts across different computers/nodes.

2. NODES COMPUTE

Each computer processes its part using its own private CPU and memory.

3. MESSAGES EXCHANGED

Nodes send results/data to each other over the network to combine the final answer.

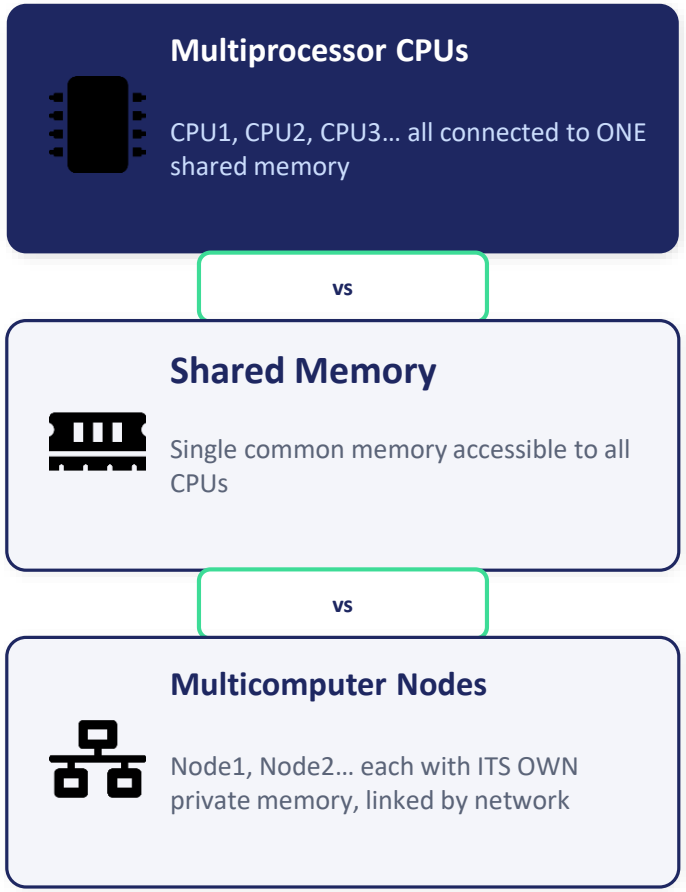
This is exactly how distributed systems, clusters, and cloud computing scale to huge workloads.

Multiprocessor vs Multicomputer — Structure

Shared memory vs private memory + network

Remember

- Multiprocessor → one OS, shared memory
- Multicomputer → many OS copies, private memory
- Communication style is the #1 distinguishing factor



Exam Tip

Draw both structures side by side (shared memory bus vs network-connected nodes) for full diagram marks.

Multiprocessor vs Multicomputer — Full Comparison

Multiprocessor System

- Shared memory, tightly coupled
- Single operating system controls all CPUs
- Fast inter-CPU communication
- Example: multi-socket server

Multicomputer System

Loosely coupled independent computers, each running its own OS and memory, communicating only through network messages.

Exam Comparison Point

Multicomputers scale to thousands of nodes (clusters/cloud) far more easily than shared-memory multiprocessors.